

Задание 1. Фрагмент «Руководства системного программиста» (Раздел 2. Структура программы)

Документ разработан в соответствии с требованиями ГОСТ 19.503-79 «Единая система программной документации. Руководство системного программиста. Требования к содержанию и оформлению».

АННОТАЦИЯ

Настоящий документ содержит фрагмент руководства системного программиста для веб-сайта кафедры информационных и коммуникационных технологий РГПУ им. А. И. Герцена. Материал ориентирован на специалистов, отвечающих за техническое сопровождение, администрирование и эксплуатацию ресурса.

2. СТРУКТУРА ПРОГРАММЫ

2.1. Общие сведения о структуре

Программный продукт представляет собой официальный сайт кафедры, работающий на базе CMS Grav. В основе архитектуры лежит flat-file подход: данные хранятся не в базе данных, а непосредственно в файловой системе. Развертывание сайта осуществляется в контейнерах Docker, оркестрация которых выполняется средствами Kubernetes. Ресурс доступен по адресу: <https://herzen.spb.ru>.

2.2. Составные части программы

Программный комплекс включает четыре ключевых логических модуля, соответствующих структуре каталогов:

1. Ядро Grav (директория /system/):

- o Обработка входящих URL и определение маршрутов.
- o Преобразование контента из Markdown в HTML с помощью библиотеки Parsedown.
- o Формирование пользовательского интерфейса через шаблонизатор Twig.
- o Многослойное кэширование на базе Doctrine Cache.
- o Управление системными событиями и работа с консолью через компоненты Symfony.

2. Пользовательское пространство (директория /user/):

- o accounts/ — данные учетных записей редакторов и администраторов (формат YAML).
- o config/ — файлы настроек ядра, плагинов и тем оформления.
- o pages/ — содержимое страниц сайта, подготовленное в разметке Markdown.
- o plugins/ — установленные дополнения, расширяющие функциональность (свыше 50 наименований).

- o themes/ict/ — текущая тема, задающая внешний вид и шаблоны отображения.

3. Компоненты интеграции с Google Cloud Platform:

- o oauth2callback.php — реализует вход пользователей через Google OAuth 2.0.
- o pr_gca_save_callback.php — принимает вебхуки от Google Forms и сохраняет данные в gca.json.
- o pr_gda_save_callback.php — связывает страницы сайта с директориями на Google Диске.
- o pr_gd_save_callback.php — отвечает за автоматическую загрузку и конвертацию файлов с Google Диска.
- o client_id.json — хранит секретные ключи для авторизации приложения в Google Cloud.

4. Инфраструктурные компоненты:

- o Dockerfile — сценарий автоматической сборки образа приложения.
- o deploy.yaml — файл манифеста Kubernetes, описывающий конфигурацию сети, служб и развертывания контейнеров (Ingress, Service, Deployment).

2.3. Связи между составными частями

Обработка запроса пользователя происходит по следующей схеме:

1. HTTP-запрос поступает на вход Kubernetes Ingress (домен ict.herzen.spb.ru).
2. Ingress направляет трафик на внутренний сервис Kubernetes Service (apps-ict), слушающий порт 80.
3. Сервис распределяет нагрузку и передает запрос в нужный Docker-контейнер, где функционирует веб-сервер и Grav CMS.
4. Ядро Grav загружает глобальные параметры из system.yaml и определяет активную тему ict.
5. На основе URL система находит соответствующий файл с контентом в папке pages/.
6. Разметка Markdown обрабатывается плагинами и преобразуется в HTML с применением шаблонов Twig из активной темы.
7. Сформированная HTML-страница возвращается пользователю в браузер.

2.4. Связи с другими программами

Для взаимодействия со сторонними системами используются стандартные протоколы:

- **Google OAuth 2.0:** HTTPS. Обеспечивает безопасную аутентификацию посетителей сайта.

- **Google Forms:** HTTP GET (Webhooks). Служит для передачи данных из форм обратной связи на сайт.
- **Google Drive API и Calendar API:** REST API / HTTPS. Применяется для синхронизации с календарем мероприятий, загрузки и конвертации документов.
- **Docker Engine и Kubernetes API:** Внутренние протоколы управления (Container API, HTTP/HTTPS). Решают задачи оркестрации, масштабирования и изоляции среды исполнения.
- **SMTP-сервер:** Протокол SMTP. Используется для рассылки системных уведомлений по электронной почте.

2.5. Организация хранения данных

Поскольку система построена на flat-file архитектуре, вся информация сохраняется на диске сервера без использования баз данных. Применяются следующие форматы:

- **Markdown (.md):** текстовый контент страниц (каталог pages/).
- **YAML (.yaml):** конфигурационные файлы сайта, настройки плагинов и профили пользователей (config/, accounts/).
- **JSON (.json / .txt):** динамические данные, результаты обработки вебхуков от Google Forms, настройки синхронизации с Google Drive и временные токены доступа.
- **LOG (.log):** журналы ошибок и предупреждений (/user/data/logerrors/).

Задание 2. Список используемого программного обеспечения и технологий

1. Базовое системное ПО и окружение

- **Linux (Ubuntu/Debian/CentOS):** операционная система хоста для запуска веб-серверов.
- **Apache / Nginx:** веб-сервер, принимающий и обрабатывающий входящие HTTP-запросы.
- **PHP (начиная с версии 7.3.6):** среда выполнения серверного кода, на которой написан движок сайта.
 - Расширения PHP: curl (обращение к API Google), json (разбор ответов), mbstring (работа с кодировками), openssl (шифрование и OAuth), gd (обработка изображений), zip (архивация).
- **Docker:** система контейнеризации, упаковывающая приложение и его зависимости в изолированный контейнер.
- **Kubernetes:** платформа оркестрации, управляющая контейнерами, обеспечивающая автоматическое масштабирование, балансировку трафика (через Nginx Ingress) и отказоустойчивость.

- **Git / GitHub:** распределенная система контроля версий для отслеживания изменений и синхронизации кодовой базы.

2. Компоненты CMS-платформы

- **Grav CMS (версия 2.x):** основная flat-file система управления содержимым.
- **Twig Templating:** компонент ядра для гибкой разработки шаблонов интерфейса.
- **Parsedown:** внутренний конвертер Markdown в валидный HTML.
- **Doctrine Cache:** модуль кэширования, ускоряющий загрузку страниц.
- **Symfony Components (Event Dispatcher, Console):** библиотеки для управления событиями системы и работы с интерфейсом командной строки.

3. Ключевые функциональные плагины Grav

- `admin / admin-addon-user-manager` — визуальная панель управления контентом и учетными записями.
- `login / form / email` — инструменты для авторизации, обработки веб-форм и отправки сообщений.
- `langswitcher` — расширение для поддержки мультиязычности (русский / английский).
- `feed` — генератор новостных RSS-лент.
- `highlight / mathjax` — модули для подсветки синтаксиса кода и рендеринга математических выражений LaTeX.
- `leaflet / lightslider` — встраивание интерактивных карт (корпуса РГПУ) и создание слайдеров.
- `events-manager` — компонент для ведения календаря событий и управления мероприятиями кафедры.
- Плагины семейства `markdown-*` (`color`, `collapsible`, `details`, `notices`) — расширяют синтаксис Markdown, позволяя добавлять цветной текст, спойлеры, иконки и информационные блоки.